

PROGRAM 1:-

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = -1;
```

```
// PUSH operation
```

```
void push(int x)
```

```
{
    if (top == MAX - 1)
    {
        printf("Stack Overflow! Cannot insert %d.\n", x);
    }
    else
    {
        top++;
        stack[top] = x;
        printf("%d pushed into stack.\n", x);
    }
}
```

```
// POP operation
```

```
void pop()
```

```
{
    if (top == -1)
    {
        printf("Stack Underflow! Stack is empty.\n");
    }
    else
    {
        printf("%d popped from stack.\n", stack[top]);
        top--;
    }
}
```

```
// PEEK operation
```

```
void peek()
```

```
{
    if (top == -1)
    {
        printf("Stack is empty.\n");
    }
    else
    {
        printf("Top element = %d\n", stack[top]);
    }
}
```

```

}

// DISPLAY operation
void display()
{
    int i;

    if (top == -1)
    {
        printf("Stack is empty.\n");
    }
    else
    {
        printf("Stack elements are: ");
        for (i = top; i >= 0; i--)
        {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}

int main()
{
    push(10);
    push(20);
    push(30);
    push(40);
    push(50);

    display();

    push(60);    // Stack Overflow

    peek();

    pop();
    pop();

    display();

    peek();

    return 0;
}

```

OUTPUT:-

10 pushed into stack.
20 pushed into stack.
30 pushed into stack.
40 pushed into stack.
50 pushed into stack.

Stack elements are: 50 40 30 20 10

Stack Overflow! Cannot insert 60.

Top element = 50

50 popped from stack.
40 popped from stack.

Stack elements are: 30 20 10

Top element = 30

Stack Overflow and Underflow:-

Stack Overflow:

When the stack is already full and we try to perform PUSH(), overflow occurs.

For MAX = 5:

10 20 30 40 50
 ↑
 TOP

Trying to insert another element produces:
Stack Overflow!

Stack Underflow:

When the stack is empty and we try to perform POP(), underflow occurs.
Stack Underflow! Stack is empty.

Operation TimeComplexity Space Complexity

PUSH	O(1)	O(1)
POP	O(1)	O(1)
PEEK	O(1)	O(1)
DISPLAY	O(n)	O(1)
Entire Stack	—	O(n)

•When the stack size is fixed, no more elements can be inserted after reaching its capacity. The program must check the top index before inserting to prevent accessing memory outside the array.

PROGRAM 2:-

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int queue[MAX];
```

```
int front = -1;
```

```
int rear = -1;
```

```
// ENQUEUE operation
```

```
void enqueue(int x)
```

```
{
```

```
    // Check whether queue is full
```

```
    if ((rear + 1) % MAX == front)
```

```
    {
```

```
        printf("Queue Overflow! Cannot insert %d.\n", x);
```

```
        return;
```

```
    }
```

```
    // First element
```

```
    if (front == -1)
```

```
    {
```

```
        front = 0;
```

```
        rear = 0;
```

```
    }
```

```
    else
```

```
    {
```

```
        rear = (rear + 1) % MAX;
```

```
    }
```

```
    queue[rear] = x;
```

```
    printf("%d inserted into queue.\n", x);
```

```
}
```

```
// DEQUEUE operation
```

```
void dequeue()
```

```
{
```

```
    if (front == -1)
```

```
    {
```

```
        printf("Queue Underflow! Queue is empty.\n");
```

```

        return;
    }

    printf("%d deleted from queue.\n", queue[front]);

    // If only one element was present
    if (front == rear)
    {
        front = -1;
        rear = -1;
    }
    else
    {
        front = (front + 1) % MAX;
    }
}

// FRONT operation
void frontElement()
{
    if (front == -1)
    {
        printf("Queue is empty.\n");
    }
    else
    {
        printf("Front element = %d\n", queue[front]);
    }
}

// DISPLAY operation
void display()
{
    int i;

    if (front == -1)
    {
        printf("Queue is empty.\n");
        return;
    }

    printf("Queue elements are: ");

    i = front;

    while (1)
    {
        printf("%d ", queue[i]);

```

```

        if (i == rear)
            break;

        i = (i + 1) % MAX;
    }

    printf("\n");
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
    enqueue(40);
    enqueue(50);

    display();

    enqueue(60);    // Queue Overflow

    frontElement();

    dequeue();
    dequeue();

    display();

    enqueue(60);
    enqueue(70);

    display();

    frontElement();

    return 0;
}

```

OUTPUT:-

10 inserted into queue.
 20 inserted into queue.
 30 inserted into queue.
 40 inserted into queue.
 50 inserted into queue.

Queue elements are: 10 20 30 40 50

Queue Overflow! Cannot insert 60.

Front element = 10

10 deleted from queue.

20 deleted from queue.

Queue elements are: 30 40 50

60 inserted into queue.

70 inserted into queue.

Queue elements are: 30 40 50 60 70

Front element = 30

Circular Queue Complexity:-

<u>Operation</u>	<u>Time Complexity</u>	<u>SpaceComplexity</u>
ENQUEUE	O(1)	O(1)
DEQUEUE	O(1)	O(1)
FRONT	O(1)	O(1)
DISPLAY	O(n)	O(1)
Entire Queue	—	O(n)

Why Circular Queue Uses Memory Better

•In a simple linear queue, suppose the array has 5 positions:

[10][20][30][40][50]
↑ ↑
FRONT REAR

After deleting 10 and 20:

[][][30][40][50]
 ↑ ↑
 FRONT REAR

•There are two empty positions at the beginning. However, if REAR has already reached the last array position, a normal linear queue cannot insert another element.

This is called false overflow or wasted space.

A circular queue solves this by allowing REAR to wrap around:

[60][70][30][40][50]
↑ ↑
REAR FRONT

Linear Queue vs Circular Queue

<u>Feature.</u>	<u>Linear Queue</u>	<u>Circular Queue</u>
Structure	Straight line	Circular
Memory utilization	Lower	Better
Reuses deleted positions	No	Yes
ENQUEUE	$O(1)$	$O(1)$
DEQUEUE	$O(1)$	$O(1)$
Space	$O(n)$	$O(n)$
False overflow	Possible	Avoided

Main advantage: A circular queue makes better use of a fixed-size array by connecting the last position back to the first position.